QFLOW: AI-POWERED SMART TOKEN AND QUEUE MANAGEMENT SYSTEM FOR DYNAMIC COUNTER ASSIGNMENT AND REAL-TIME OPERATIONS

A PROJECT REPORT

Submitted by

ADITHYA P K

(Registration No: 310624243009)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY,

CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled "**QFLOW: AI-POWERED SMART TOKEN AND QUEUE MANAGEMENT SYSTEM FOR DYNAMIC COUNTER ASSIGNMENT AND REAL-TIME OPERATIONS**" is the Bonafide work of "**ADITHYA P K (310624243009)**", who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. T. Kalai Selvi

Easwari Engineering College,

Ramapuram, Chennai – 600089.

SUPERVISOR

Ms. D. S. K. Harshitha

Easwari Engineering College,

Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate Of Viva-Voce Examination

This is to certify that the Viva-Voce Examination of the Project Work entitled "QFLOW: AI-POWERED SMART TOKEN AND QUEUE MANAGEMENT SYSTEM FOR DYNAMIC COUNTER ASSIGNMENT AND REAL-TIME OPERATIONS" submitted by Mr. Adithya P K (Reg. No. 310624243009) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology was conducted on 23-10-2026 in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date:

ABSTRACT

Queueing environments across commercial and public service institutions—including banking branches, outpatient clinics, customer care centers, and administrative offices—routinely experience severe customer congestion, unpredictable waiting durations, and inefficient teller resource utilization. Conventional queue management solutions rely on thermal paper token dispensers, fixed static display monitors, and inflexible First-Come-First-Served (FCFS) counter assignments.

To address these systemic bottlenecks, this project presents QFlow, an AI-powered, full-stack, real-time smart token and queue management system designed for responsive customer self-service and dynamic counter orchestration. Developed on the MERN stack (MongoDB, Express.js, React 19, Node.js) with native Socket.IO bi-directional WebSocket communication, QFlow replaces physical hardware kiosks with an accessible browser-based mobile portal. Customers join the virtual queue on their personal mobile devices by scanning a physical QR code encoding the local gateway IP address, requiring no mobile app installation or account registration. Upon token issuance, a lightweight multi-factor predictive engine estimates the customer's anticipated service duration and queue waiting time using a calibrated synthetic historical baseline adjusted dynamically for service complexity, time-of-day peak multipliers, and active queue length. Simultaneously, an automated heuristic counter recommendation algorithm scores and assigns incoming customers to optimal service counters based on live counter occupancy, pending workload, and estimated time-to-completion.

Staff operators manage the queue via a dedicated control console that supports calling next, starting service, completing consultations, marking unserved customers as skipped, and transferring tokens across departments. Real-time token lifecycle events trigger instant browser push notifications and haptic device vibration alerts on the customer's mobile device when their token is called. Comprehensive functional and concurrency testing demonstrates that the end-to-end architecture executes atomic database transitions, isolates customer subscriptions into dedicated socket channels, which prevents race conditions.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. T. KalaiSelvi**, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. D. S. K. Harshitha**, Department of Artificial Intelligence and Data Science for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.1.1 Need for a Centralized Queue and Token Management System	1
	1.1.2 Limitations of Existing Manual and Physical Token Systems	1
	1.2 OBJECTIVE	2
	1.3 SCOPE OF THE PROJECT	3
	1.4 MOTIVATION	4
	1.5 ORGANIZATION OF THE REPORT	5
2	**RELATED WORK**	**6**
	2.1 LITERATURE REVIEW	6
	2.2 EXISTING SYSTEMS	7
	2.3 INFERENCE FROM LITERATURE REVIEW	9
	2.4 EXTRACTION FROM LITERATURE REVIEW	9
3	**SYSTEM ANALYSIS**	**11**
	3.1 PROBLEM DEFINITION	11
	3.2 PROPOSED SOLUTION	11
	3.2.1 Functional Requirements	12
	3.2.2 Non-Functional Requirements	14
	3.3 SOFTWARE COMPONENTS	14
	3.4 HARDWARE COMPONENTS	16

CHAPTER NO.	TITLE	PAGE No.
	3.5 USE CASES	16
4	**METHODOLGY & IMPLEMENTATION**	**20**
	4.1 SYSTEM ARCHITECTURE AND EXPERIMENTAL SETUP	20
	4.1.1 High-Level System Architecture	20
	4.1.2 System Deployment and Network Topology	23
	4.2 MODULE IMPLEMENTATION DETAILS	24
	4.2.1 Data Source Module	25
	4.2.2 Steps of Algorithm	26
	4.2.3 Data Processing Module	27
	4.2.4 Explainability Module	28
	4.2.5 Visualization and Dashboard Module	28
	4.3 CLASS DIAGRAM / DATA MODEL	28
	4.4 SEQUENCE DIAGRAM	30
	4.5 TESTING AND RESULTS	31
5	**CONCLUSION AND FUTURE ENHANCEMENTS**	**35**
	5.1 CONCLUSION	35
	5.2 FUTURE ENHANCEMENTS	36
	APPENDIX 1: SAMPLE CODE	**37**
	1. Project Directory Structure	37
	2. Installation and Execution Commands	38
	3. Server Environment Configuration	38
	4. Database Models and Schema Architecture	38
	5. Main Server Entry Point and WebSocket Setup	39
	6. Token Controller and Atomic Number Generation	40
	7. Authentication Controller and JWT Middleware	41
	8. AI Prediction Engine and Duration Calculation	42
	9. Dynamic Counter Recommendation Scoring	43

CHAPTER NO.	TITLE	PAGE No.
	10. Socket.IO Real-Time Multiplexing and Broadcasts	43
	PROJECT SCREENSHOTS	**46**
	REFERENCES	**56**

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE No.
2.1	Comparison of Existing Approaches to Queue and Token Management	8
2.2	Design Principles Extracted from Literature and Their Use in QFlow	9
3.1	Functional Requirements of QFlow	12
3.2	Software Components of QFlow	14
3.3	Hardware Components for Development and Deployment	16
3.4	Description of Main Use Cases	17
4.1	REST API Endpoints of QFlow	21
4.2	Experimental Setup Used for Building and Testing	24
4.3	Summary of QFlow Modules	24
4.4	Service Catalog and Base Parameters Accepted by the Data Source Module	25
4.5	MongoDB Collections Used by QFlow	28
4.6	Functional Test Cases and Results	31
4.7	Technical Observations and Edge Cases Observed During Testing	33

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE No.
3.1	Use Case Diagram of QFlow	19
4.1	System Architecture of QFlow	20
4.2	System Deployment and Network Topology of QFlow	23
4.3	Class Diagram / Entity-Relationship Schema of the QFlow Data Model	28
A.1	Staff and Administrative Authentication Portal	46
A.2	Live Staff Operations Dashboard with Active Counters	47
A.3	Customer QR Code Registration Poster and Modal	47
A.4	Mobile Customer Portal - Welcome and Value Proposition	48
A.5	Mobile Customer Registration Form with Service Selection	49
A.6	Mobile Service Catalogue Modal Dialog	50
A.7	Live Mobile Customer Ticket - Waiting Status with AI Estimation	51
A.8	Real-Time Mobile Haptic & Visual Turn Alert Screen	52
A.9	Active Mobile Consultation In-Service Status Monitor	53
A.10	Service Completion Confirmation Screen	54
A.11	Counter Management and Dynamic Fleet Allocation Workspace	54
A.12	Real-Time Queue Inspection and Live Token Roster	55
A.13	Executive Operational Analytics Dashboard with Traffic Distributions	55
A.14	Active Counter Workspace Telemetry Detail Cards	55

CHAPTER 1
INTRODUCTION

This chapter deals with the introduction, operational problem definition, engineering objectives, scope boundaries, motivational framework, and organizational structure underpinning the development of QFlow, an AI-powered smart token and queue management system.

1.1 OVERVIEW

1.1.1 Need for a Centralized Queue and Token Management System

High-density service facilities such as retail banking branches, outpatient medical clinics, municipal government offices, and telecommunication support centers experience fluctuating customer arrival volumes throughout the operating day. When arrival rates exceed immediate service capacity, queues inevitably materialize. In the absence of an automated management architecture, customer flow degenerates into physical crowding, ambiguous service priorities, and heightened operational anxiety for both visitors and service staff.

A centralized queue and token management platform introduces structured discipline to this operational dynamic. By transitioning customer entry into an atomic, sequential digital token allocation model, the system guarantees strict fairness and chronological transparency. Tellers are relieved from manual queue arbitration, enabling them to focus entirely on customer consultation. Furthermore, facility supervisors gain instant visibility into live floor occupancy, pending backlogs, and counter productivity, transforming an unpredictable physical environment into a measured, manageable operational workflow.

1.1.2 Limitations of Existing Manual and Physical Token Systems

Conventional approaches to customer queue coordination suffer from distinct functional limitations:

- **Hardware Vulnerability and Maintenance Overhead:** Physical kiosk dispensers rely on mechanical button triggers, thermal printers, and continuous paper rolls.

Mechanical jams and paper exhaustion frequently halt token issuance during peak operational hours.

- **Spatial Tethering and Waiting Lounge Congestion:** Customers receiving paper slips are tethered to physical waiting lounges because turn announcements are conveyed solely via ceiling-mounted LED numeric displays or overhead audio buzzers.
- **Absence of Contextual Duration Telemetry:** Printed tokens display only a sequential number without providing actionable estimates of expected waiting duration, leaving visitors uninformed regarding their anticipated consultation time.
- **Static and Rigid Counter Allocation:** Legacy hardware systems assign tokens to fixed service counters based on static First-Come-First-Served (FCFS) rules, preventing dynamic load rebalancing when specific consultations overrun.
- **Data Siloing and Telemetry Deficits:** Manual registers and standalone electronic systems fail to aggregate operational metrics, depriving management of historical data required for strategic staff scheduling.

1.2 OBJECTIVE

The primary objective of QFlow is to design and develop an intelligent, real-time, browser-accessible queue management platform that replaces physical ticketing hardware with a mobile-first digital workflow. The specific objectives are:

1. **Responsive Mobile Self-Service:** To engineer a mobile web portal accessible via a dynamic QR code without requiring native application downloads or user account registration.
2. **Atomic Token Sequencing:** To provide atomic sequential token generation (e.g., A101, A102) linked to specific service categories with zero collision risk under concurrent requests.
3. **AI-Powered Wait Estimation:** To construct an in-memory predictive engine utilizing historical synthetic data to forecast transaction durations and personalized waiting times.

4. **Dynamic Counter Recommendation:** To implement a heuristic scoring algorithm that evaluates counter status, queue backlogs, and remaining service times to recommend optimal routing.
5. **Staff Operations Dashboard:** To build a centralized workstation console providing counter status switching, next-token calling, service starting, completion recording, skipping, and inter-counter transferring.
6. **Sub-Millisecond Real-Time Push:** To establish bi-directional real-time event dissemination using Socket.IO to deliver visual notifications and haptic vibration alerts directly to customer smartphones.
7. **Operational Telemetry Analytics:** To compute aggregate facility metrics including hourly queue distributions, average wait times, and teller throughput.
8. **Multi-Container Architecture:** To package the full-stack system using a modular architecture for straightforward local and on-premise deployment.

1.3 SCOPE OF THE PROJECT

The scope of QFlow encompasses an end-to-end software platform for managing customer queues, service desk allocations, token lifecycle transitions, and real-time operational analytics.

Functional Scope (In-Scope):

- **Dynamic QR Generation:** Automated detection of the host's Wi-Fi network gateway IPv4 address for multi-device cross-network accessibility.
- **Frictionless Mobile Registration:** Zero-install web form capturing name, contact number, and selected service category.
- **State Machine Lifecycle:** Atomic transitions across WAITING, CALLED, IN_SERVICE, COMPLETED, SKIPPED, and TRANSFERRED states.
- **Predictive Duration Modeling:** Synthetic historical baseline regression combined with temporal peak multipliers and queue depth adjustments.
- **Channel-Isolated WebSockets:** Socket.IO room architecture separating global queue telemetry ('queue') from private customer notifications ('token:{id}').

- **Haptic Multisensory Notification:** HTML5 Web Vibration API execution directly on customer handsets upon counter calling.
- **Administrative Telemetry:** MongoDB aggregation pipelines generating real-time traffic distributions and teller productivity reports.
- **Role-Based Access Control:** Secure staff and supervisor authentication utilizing JSON Web Tokens (JWT) and bcrypt password hashing.

Operational Boundaries (Out-of-Scope):

- **Physical Hardware Interfacing:** Direct RS-485 serial communication with physical seven-segment LED displays or legacy thermal paper printers.
- **Cellular SMS Telephony:** Paid SMS gateway integration, relying instead on native browser WebSockets and push notifications.
- **Biometric Customer Verification:** Facial recognition or fingerprint authentication for incoming visitors.
- **Multi-Tenant Enterprise Cloud:** Multi-site global clustering; current scope targets single-facility on-premise or campus deployment.

1.4 MOTIVATION

Queueing environments are ubiquitous across modern society. Unpredictable waiting durations degrade customer satisfaction and increase stress for service personnel. The widespread adoption of smartphones equipped with high-resolution cameras presents an opportunity to transform physical queueing into an accessible digital self-service interaction. By allowing visitors to scan a QR code and receive live updates and haptic alerts on their own phones, physical congestion in waiting rooms is effectively mitigated.

As an academic project in Artificial Intelligence and Data Science, QFlow addresses real-world systems engineering challenges: asynchronous client-server architectures, real-time WebSocket state synchronization, atomic document locking under concurrency, and mathematical modeling of service queue dynamics. The resulting system demonstrates how standard web technologies can eliminate expensive kiosk hardware while improving customer experience.

1.5 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows:

- **Chapter 2 - Related Work:** Reviews foundational queueing theory, real-time distributed web communication protocols, state-of-the-art queue architectures, and derives the design principles governing QFlow.
- **Chapter 3 - System Analysis:** Formalizes the problem definition, details functional and non-functional requirements, itemizes software dependencies and hardware development environments, and outlines primary user use cases.
- **Chapter 4 - Implementation:** Explains system architecture, deployment topology, core algorithmic formulations for AI duration and counter recommendation, Mongoose database schemas, and empirical test results.
- **Chapter 5 - Conclusion and Future Enhancements:** Summarizes key project achievements, identifies current operational boundaries, and proposes directions for subsequent development.
- **Appendix 1 - Sample Code & Screenshots:** Provides representative source code excerpts, followed by Project Screenshots and References.

CHAPTER 2
RELATED WORK

This chapter deals with a critical literature survey of queueing management paradigms, analyzing conventional physical token systems, documenting operational bottlenecks in legacy customer service facilities, and formulating the proposed QFlow real-time predictive architecture.

2.1 LITERATURE REVIEW

Queueing Theory and Operational Dynamics. Queueing theory, initiated by A. K. Erlang's pioneering mathematical formulations for telephone exchange congestion in 1909 [1], provides the quantitative foundation for evaluating waiting lines. Kendall's notation classifies queueing systems based on arrival patterns, service time distributions, and the number of servers (e.g., M/M/1, M/M/c) [2]. Little's Law ($L = \lambda W$) mathematically establishes that the average number of customers in a stationary queueing system (L) equals the long-term average arrival rate (λ) multiplied by the average time a customer spends in the system (W) [3]. While classical analytical models assume idealized Poisson arrival processes and exponential service distributions, real-world service facilities experience dynamic variance driven by human behavioral factors and transaction complexity. QFlow adopts these theoretical principles by dynamically computing estimated wait time as the aggregated pending workload divided by active counter capacity.

Predictive Scheduling in Service Operations. Larson's seminal work on the psychology of queueing demonstrated that perceived waiting time, lack of temporal visibility, and unexplained delays exert a significantly more detrimental impact on customer satisfaction than objective elapsed time [4]. Providing customers with an accurate initial duration estimate fundamentally reduces perceived anxiety. Modern predictive frameworks increasingly apply machine-learning heuristics to forecast operational service durations using features such as transaction category, temporal peak windows, and real-time server density [5]. QFlow incorporates these insights via a lightweight weighted-regression model that calibrates base service metrics against historical peak variances.

Real-Time Bi-Directional Web Architectures. Traditional web applications rely on stateless HTTP request-response cycles. Under high-frequency polling architectures where clients query the server every few seconds to check queue position, server thread pools become rapidly saturated by redundant HTTP handshakes [6]. The WebSocket protocol (IETF RFC 6455) provides full-duplex, persistent communication channels over a single TCP connection, drastically minimizing overhead and latency [7]. Socket.IO abstracts WebSocket transport with automatic HTTP long-polling fallback, structured event multiplexing, and room-based channel isolation [8]. QFlow leverages Socket.IO rooms to broadcast public queue states while privately channeling status alerts to specific customer devices.

Document-Oriented Database Concurrency. In transactional queueing systems, concurrency control is vital. When multiple service counters attempt to call the next waiting customer simultaneously, naive read-modify-write patterns result in race conditions where two tellers are assigned the same ticket [9]. Modern document stores like MongoDB provide atomic single-document modifications via operators such as findOneAndUpdate, ensuring linearizable state transitions without the operational overhead of heavy multi-table SQL locks [10].

2.2 EXISTING SYSTEMS

Current queue management approaches deployed across public and commercial facilities can be categorized into four primary paradigms:

1. **Disconnected Spreadsheets and Paper Registers:** Activities are logged manually in paper notebooks or spreadsheets. While simple, they fail to provide real-time coordination, lack customer-facing displays, and suffer from high human error rates.
2. **Dedicated Self-Service Hardware Kiosks:** Freestanding touchscreen terminals equipped with thermal printers. While functional, they entail high acquisition costs, continuous paper consumable overhead, and still produce physical slips without smartphone integration.

3. **Proprietary Native Mobile Applications:** Commercial enterprise apps requiring customers to install native software and register user accounts before joining a line. This friction causes many casual visitors to bypass the system.
4. **QFlow (Proposed System):** A zero-install, browser-based web architecture. Customers scan a physical QR code using their existing phone camera, instantly joining the queue on a mobile browser with AI-driven wait estimation, dynamic counter scoring, Socket.IO haptic alerts, and complete administrative queue governance.

Table 2.1: Comparison of Existing Approaches to Queue and Token Management

Approach	Physical Hardware	Mobile Integration	Wait Estimation	Real-Time Push	Friction
Paper Thermal Dispensers	High (Printers, Displays)	None	None	Audio chime only	Low friction, high confinement
Enterprise Touchscreen Kiosks	High (Costly kiosks)	Minimal (Optional SMS)	Static average	Public display screens	Moderate friction, hardware maintenance
Proprietary Native Apps	Low	High	Basic heuristic	Push notifications	Very high friction (App install + signup)
QFlow (This Project)	Minimal (Static printed QR)	High (Zero-install web app)	Dynamic AI prediction	Socket.IO push + Phone vibration	Zero friction (Scan and go)

2.3 INFERENCE FROM LITERATURE REVIEW

The comprehensive literature review leads to the following key inferences:

- **Visibility Reduces Perceived Waiting Time:** Accurately calculating and displaying expected wait times alleviates customer stress and prevents queue abandonment [4].
- **Frictionless Onboarding is Critical:** Systems that mandate account creation or native application installation encounter steep adoption resistance from one-time visitors. A browser-based QR entry model minimizes barriers to entry.
- **Push-Based Updates Outperform Polling:** Continuous HTTP polling degrades mobile battery life and overburdens server resources. Persistent WebSocket connections provide instant notification with minimal network overhead [6], [7].
- **Atomic State Transitions Prevent Collisions:** Tellers operating under peak loads require guaranteed atomic database operations to eliminate double-calling race conditions [10].
- **Dynamic Counter Allocation Improves Utilization:** Static counter assignment leads to uneven queue distribution; automated scoring balances traffic according to live desk capacity.

2.4 EXTRACTION FROM LITERATURE REVIEW

Based on these inferences, the architectural and algorithmic design principles summarized in Table 2.2 were extracted and implemented in QFlow.

Table 2.2: Design Principles Extracted from Literature and Their Use in QFlow

Principle (Literature Source)	Technical Implementation in QFlow
Frictionless Mobile Access [4]	Dynamic QR generation embedding local network IP; zero-install mobile web portal in React 19.
Dynamic Wait-Time Estimation [1], [3]	Algorithmic prediction combining service base durations, peak multipliers, and active counter counts.

Principle (Literature Source)	Technical Implementation in QFlow
Sub-Millisecond Real-Time Push [7], [8]	Socket.IO room architecture isolating global queue events ('queue') from private customer notifications ('token:{id}').
Haptic Multisensory Notification [4]	HTML5 Web Vibration API (navigator.vibrate([300, 100, 300])) triggered automatically on customer device upon token call.
Atomic Concurrency Guarantee [10]	MongoDB atomic findOneAndUpdate queries targeting status: 'WAITING' to eliminate race conditions.
Dynamic Counter Allocation [5]	100-point heuristic scoring algorithm evaluating counter status, queue backlogs, and time-to-completion.
Stateful Aggregation Analytics [2]	Mongoose pipeline calculations deriving hourly traffic distribution, average wait times, and counter throughput.

CHAPTER 3
SYSTEM ANALYSIS

This chapter deals with the comprehensive system requirements analysis, operational constraints, and technical specifications, detailing the functional and non-functional requirements alongside the hardware and software environments necessary for developing and deploying QFlow.

3.1 PROBLEM DEFINITION

Public and commercial customer service facilities frequently suffer from systemic congestion, unpredictable wait times, and poor communication between customers and staff. The specific operational problems can be stated as follows:

1. **Lack of Customer Situational Awareness:** Customers waiting for service have no reliable estimate of how long their consultation will take or how many customers ahead have complex requirements.
2. **High Infrastructure and Maintenance Costs:** Conventional queue systems depend on specialized thermal ticket dispensers, roll paper refills, mechanical buttons, and physical seven-segment counter display hardware.
3. **Physical Confinement and Crowding:** Because token notifications are communicated solely through ceiling-mounted LED screens or overhead speakers, customers are forced to remain physically packed inside waiting rooms.
4. **Inefficient Counter Workload Balancing:** When tellers manually choose which token to call or operate under static ticket queues, workloads become unevenly distributed across staff members.
5. **Absence of Centralized Operational Telemetry:** Facility managers lack actionable, real-time insights into average service durations, hourly arrival spikes, and teller idle times.

3.2 PROPOSED SOLUTION

The proposed solution is QFlow, an integrated, three-tier, real-time web application built on the MERN stack. Its architectural capabilities include:

1. **Zero-Install Customer Portal:** Accessible on any smartphone browser by scanning a dynamic QR code. Customers register their name, phone number, and requested service, immediately receiving an atomic digital token (e.g., A101).
2. **AI-Powered Duration & Wait Estimation:** An in-memory prediction service evaluates service complexity, current hour peak factors, and queue depth to generate personalized wait-time estimates.
3. **Dynamic Counter Assignment Engine:** A heuristic algorithm evaluates active counter states and assigns incoming customers to optimal counters to prevent bottlenecking.
4. **Real-Time Staff Operations Dashboard:** An administrative console allowing tellers to call the next customer, start consultations, complete services, skip absent visitors, and transfer tokens between departments.
5. **Bi-Directional WebSocket Notifications:** Socket.IO pushes live queue transitions to both staff consoles and customer smartphones, triggering visual banners and haptic phone vibrations.
6. **Operational Performance Analytics:** MongoDB aggregation pipelines generate live charts tracking hourly customer volumes, counter throughput, and average wait times.

3.2.1 Functional Requirements

Table 3.1: Functional Requirements of QFlow

ID	Requirement	Description
FR-01	User Authentication	Staff and administrative personnel can securely authenticate using email and password, receiving a signed JSON Web Token (JWT).
FR-02	QR Code Generation	The server detects the active Wi-Fi LAN IPv4 address and dynamically generates high-resolution QR codes linking directly to the mobile portal.

ID	Requirement	Description
FR-03	Customer Token Issuance	Customers can submit their name, phone number, and service category via the mobile portal to obtain an atomic sequential token (e.g., A101).
FR-04	AI Wait-Time Prediction	The system automatically calculates predicted service duration and estimated queue waiting time upon token generation.
FR-05	Dynamic Counter Recommendation	The system calculates a multi-factor score (0–100) for active counters and recommends optimal routing for incoming tokens.
FR-06	Staff Queue Management	Staff can execute token state transitions: Call Next, Start Service, Complete Service, Skip Token, and Transfer Token.
FR-07	Real-Time Event Broadcasting	Token creations, calls, and completions are broadcast instantly across Socket.IO rooms ('queue' and 'token:{id}').
FR-08	Haptic & Visual Customer Alerts	When a token status transitions to CALLED, the customer’s phone displays an on-screen alert and triggers haptic vibration pulses.
FR-09	Counter Administration	Administrators can dynamically add new counters, update staff assignments, toggle counter availability, and delete inactive desks.
FR-10	Operational Analytics & Metrics	The system aggregates completed token records into real-time visual charts showing hourly traffic, average wait times, and counter throughput.
FR-11	Database Seeding & Recovery	The system includes automated database seed scripts to populate demo accounts, standard service catalogues, and historical analytics.

3.2.2 Non-Functional Requirements

The non-functional performance, reliability, and security constraints governing QFlow include:

- **NFR-01 Real-Time Latency:** Queue state transitions must be dispatched across Socket.IO channels and rendered on client displays within 200 milliseconds of database commitment.
- **NFR-02 Concurrency & Data Integrity:** The backend must execute atomic database modifications using findOneAndUpdate to prevent race conditions during concurrent Call Next operations.
- **NFR-03 Zero-Friction Usability:** The customer interface must function seamlessly across modern mobile browsers (iOS Safari, Android Chrome) without requiring app store installation or login credentials.
- **NFR-04 Security & Password Protection:** Passwords must be hashed using bcrypt (salt rounds = 10), and all administrative endpoints must enforce JWT bearer token validation.
- **NFR-05 High Availability & Fallback:** The backend must support both configured persistent MongoDB connections and zero-config embedded MongoMemoryServer fallbacks for standalone demonstrations.

3.3 SOFTWARE COMPONENTS

Table 3.2: Software Components of QFlow

Layer	Component	Version / Specification	Purpose in QFlow
Runtime	Node.js	v18.0.0 or higher (v22.x LTS)	Asynchronous backend JavaScript execution environment
Frontend Framework	React	^19.0.0	Component-based reactive user interface rendering

Layer	Component	Version / Specification	Purpose in QFlow
Build Tool	Vite	^6.0.0 / ^8.0.0	High-performance frontend dev server and production bundler
Styling	Tailwind CSS	^3.4.0 / ^4.0.0	Utility-first responsive CSS styling for mobile and desktop
Web Framework	Express.js	^4.19.0	RESTful API routing, middleware chaining, and HTTP server
Database & ODM	MongoDB & Mongoose	MongoDB 7.0 / Mongoose ^8.3.0	Persistent document store for tokens, counters, users, and services
Real-Time Transport	Socket.IO	^4.7.5	Full-duplex WebSocket communication and room-based event dispatch
Security & Auth	jsonwebtoken & bcryptjs	JWT ^9.0.2 / bcryptjs ^2.4.3	Stateless token-based authentication and adaptive password hashing
Charts & Visualization	Recharts & Lucide React	Recharts ^2.12.0 / Lucide ^0.469.0	Analytical data graphs and iconography
QR Code Generation	qrcode & qrcode.react	qrcode ^1.5.3 / qrcode.react ^4.2.0	Server-side image generation and client QR rendering

3.4 HARDWARE COMPONENTS

Table 3.3: Hardware Components for Development and Deployment

Component	Development Workstation	Deployment Host (Server)	Client Devices (Staff & Customer)
Processor	4-Core x86-64 / ARM64 CPU	2 or more vCPUs (x86-64 / ARM64)	Any modern smartphone or desktop processor
RAM	8 GB DDR4 minimum (16 GB recommended)	4 GB RAM minimum	2 GB RAM minimum on mobile devices
Storage	256 GB SSD (500 MB free for project)	20 GB available SSD storage	Standard mobile flash storage
Network Interface	Wi-Fi (802.11ac) or Gigabit Ethernet	Wi-Fi / LAN with static or known IPv4	Wi-Fi or Cellular data connected to same subnet
Display	1920x1080 Full HD Monitor	Headless server / Standard monitor	Smartphone display (min 375px width)

3.5 USE CASES

The functional interactions of QFlow span three primary user roles:

1. **Customer:** Scans facility QR code, submits registration form, tracks queue position, receives visual and haptic alerts, and attends service counter.
2. **Staff / Teller:** Logs in with email/password, activates workstation counter, calls next customer, begins service, completes consultation, marks skips, or transfers tokens.
3. **Administrator / Supervisor:** Manages counter fleet configurations, configures service categories, inspects real-time queue rosters, and analyzes facility throughput charts.
4. **System / Automated Engine:** Performs background wait-time regression calculations, evaluates heuristic counter fitness, and dispatches multi-room WebSocket broadcasts.

Table 3.4: Description of Main Use Cases

Use Case ID	Use Case Name	Primary Actor	Pre-conditions	Main Success Scenario
UC-01	Generate QR Code	Administrator / Server	Server online on local Wi-Fi	Server detects LAN IP and generates scannable QR code
UC-02	Join Virtual Queue	Customer	Customer scans QR code on phone	Customer submits form; server issues atomic token with AI wait estimate
UC-03	Track Live Token	Customer	Token generated and active	Customer observes real-time position; phone vibrates when token is called
UC-04	Staff Login	Staff / Admin	Valid credentials in database	Staff submits email/password; server returns signed JWT token
UC-05	Call Next Customer	Staff Teller	Teller counter is AVAILABLE	Teller clicks Call Next; server atomically assigns next WAITING token
UC-06	Conduct Consultation	Staff Teller	Token is in CALLED status	Teller clicks Start Service; timer records consultation start time
UC-07	Complete Service	Staff Teller	Token is IN_SERVICE	Teller clicks Complete; consultation duration logged; counter made AVAILABLE

Use Case ID	Use Case Name	Primary Actor	Pre-conditions	Main Success Scenario
UC-08	Skip Absent Customer	Staff Teller	Called customer fails to arrive	Teller marks token SKIPPED; queue order updates live for waiting users
UC-09	Transfer Token	Staff Teller	Token requires secondary desk	Teller selects target counter; token re-routed without losing seniority
UC-10	Manage Counters	Administrator	Admin authenticated via JWT	Admin creates, activates, pauses, or deletes counter desks in fleet
UC-11	View Analytics	Admin / Supervisor	Admin authenticated via JWT	Supervisor views charts of hourly traffic, wait times, and teller output

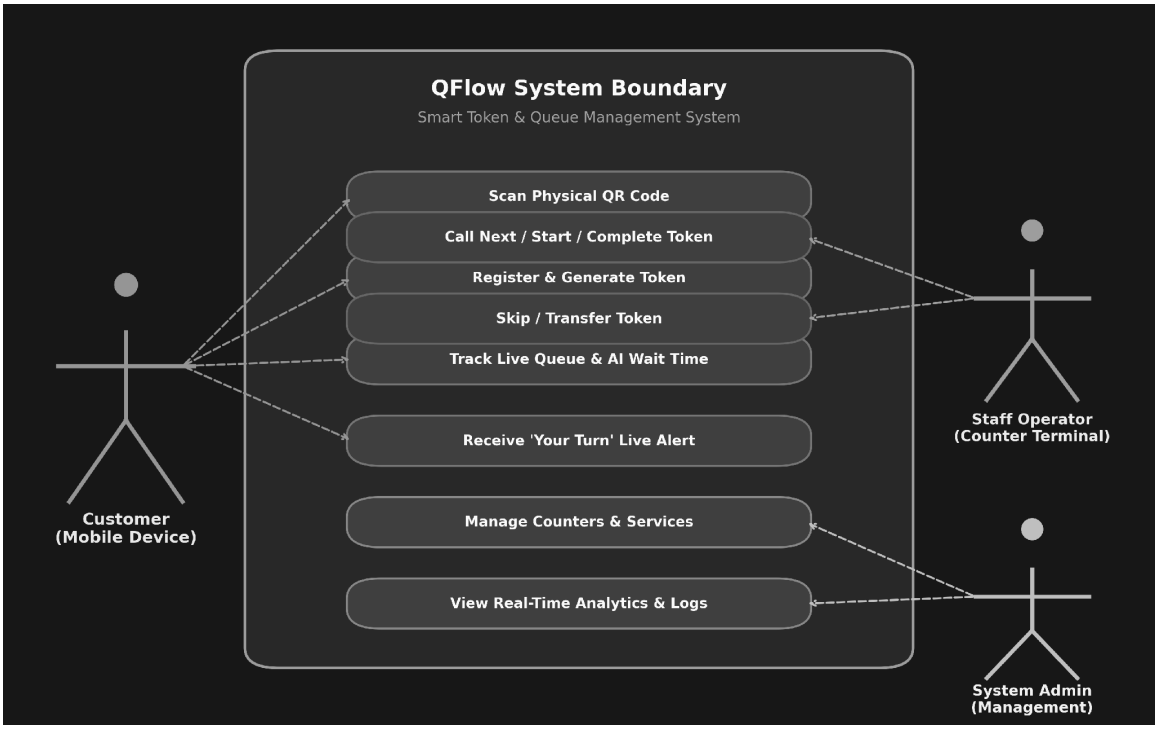


Figure 3.1: Use Case Diagram of QFlow

CHAPTER 4
METHODOLOGY & IMPLEMENTATION

This chapter deals with the core engineering methodology, module implementation details, architectural framework, class data models, sequence workflows, and empirical testing results of the QFlow platform.

4.1 SYSTEM ARCHITECTURE AND EXPERIMENTAL SETUP

4.1.1 High-Level System Architecture

QFlow follows a three-layer client-server architecture designed for high responsiveness, real-time bidirectional communication, and clean separation of concerns. The layered architecture is illustrated in Figure 4.1.

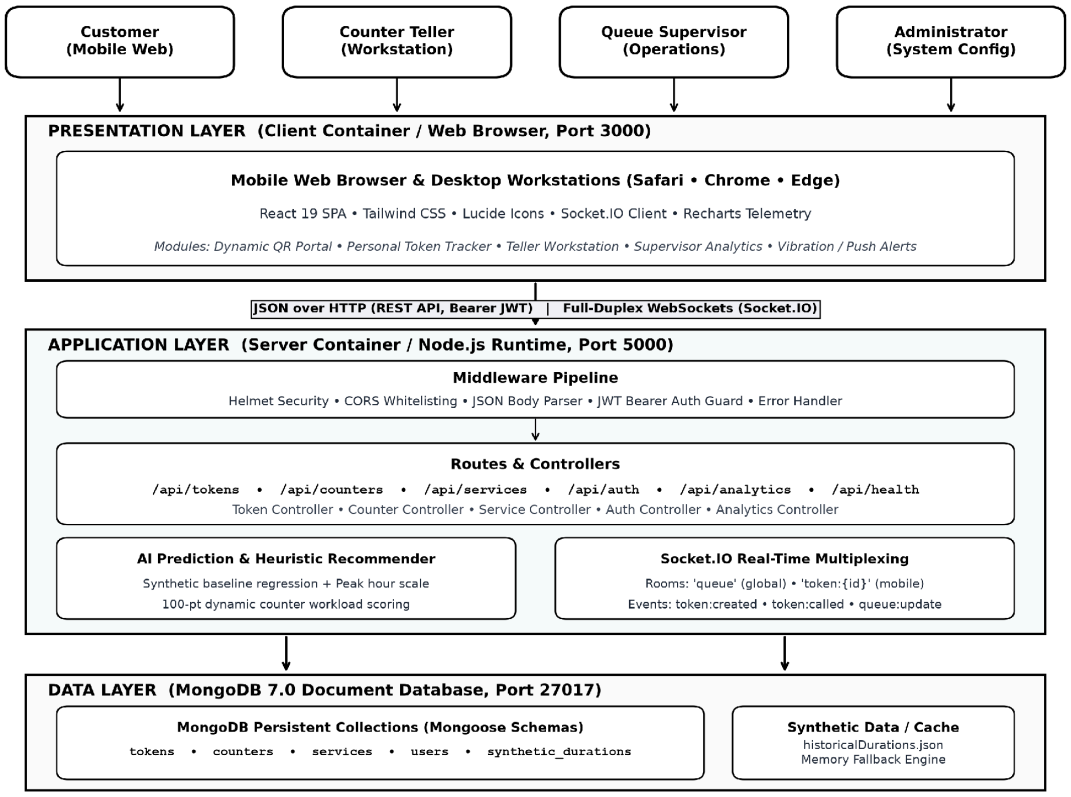


Figure 4.1: System Architecture of QFlow

The primary responsibilities of each architectural tier are structured as follows:

- **Presentation Layer (Client Container / Web Browser, Port 3000):** Constructed using React 19, Vite, and Tailwind CSS. Provides a responsive mobile self-service portal for customers and an administrative desktop workstation for staff tellers and supervisors. Incorporates Socket.IO client listeners, Recharts telemetry visualizations, Lucide icons, and the HTML5 Web Vibration API for haptic alert dispatch.
- **Application Layer (Server Container / Node.js Runtime, Port 5000):** Powered by Node.js and Express.js. Handles CORS whitelisting, JSON parsing, rate limiting, and JWT bearer authentication. Orchestrates REST API controllers (/api/tokens, /api/counters, /api/services, /api/auth, /api/analytics), manages Socket.IO WebSocket rooms ('queue', 'token:{id}'), and executes the AI duration prediction and heuristic counter assignment engines.
- **Data Layer (MongoDB 7.0 Document Store, Port 27017):** Stores persistent operational records across dedicated collections (tokens, counters, services, users). Provides linearizable atomic document updates via findOneAndUpdate, guaranteeing concurrency safety during simultaneous staff counter calls.

Table 4.1: REST API Endpoints of QFlow

HTTP Method	API Endpoint	Access Level	Functional Purpose
POST	/api/auth/login	Public	Authenticates staff/admin credentials and issues signed JWT token
GET	/api/auth/me	Authenticated	Retrieves profile and role permissions of current user session
POST	/api/tokens	Public	Creates atomic sequential token with AI duration & wait estimates
GET	/api/tokens/:id	Public	Retrieves current status, position, and wait time for a single token

HTTP Method	API Endpoint	Access Level	Functional Purpose
PATCH	/api/tokens/:id/call	Authenticated	Atomically transitions token to CALLED and alerts customer handset
PATCH	/api/tokens/:id/start	Authenticated	Transitions token to IN_SERVICE and stamps consultation start time
PATCH	/api/tokens/:id/complete	Authenticated	Completes service, logs actual duration, and frees counter desk
PATCH	/api/tokens/:id/skip	Authenticated	Marks absent customer as SKIPPED and updates queue order
PATCH	/api/tokens/:id/transfer	Authenticated	Transfers active token to another target counter without re-queuing
GET	/api/counters	Public / Auth	Retrieves all registered counters with live occupancy and status
POST	/api/counters	Admin	Registers a new service counter into the facility fleet
PATCH	/api/counters/:id/status	Authenticated	Updates counter operational state (AVAILABLE, OFFLINE)
GET	/api/services	Public	Fetches active facility service catalogue with standard base times
GET	/api/queue/active	Authenticated	Fetches complete snapshot of all active tokens in queue
GET	/api/analytics/summary	Authenticated	Generates aggregate metrics (average wait time, traffic trends)

HTTP Method	API Endpoint	Access Level	Functional Purpose
GET	/api/health	Public	Returns system uptime, active LAN IP, and customer QR portal URL

4.1.2 System Deployment and Network Topology

The physical and containerized deployment architecture of QFlow across the host server, local area network, and heterogeneous client devices is shown in Figure 4.2. The system operates seamlessly both in containerized Docker Compose environments and native Node.js runtime configurations.

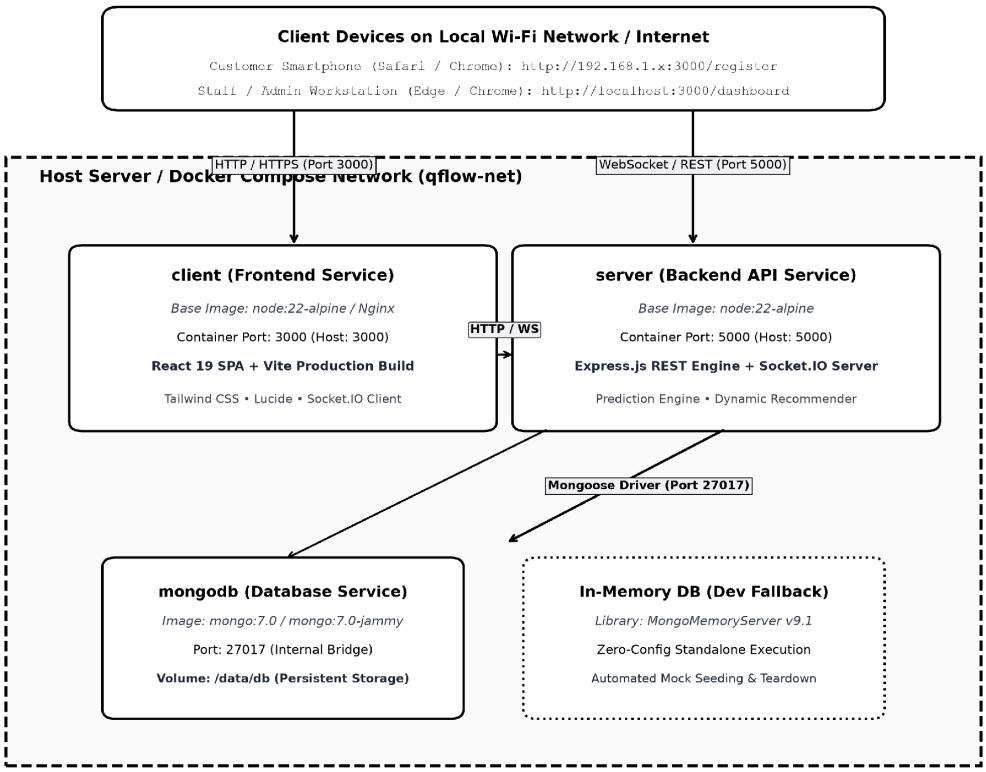


Figure 4.2: System Deployment and Network Topology of QFlow

Table 4.2: Experimental Setup Used for Building and Testing

Parameter	Development & Test Specification
Operating System	Windows 11 Enterprise (64-bit, Build 26100) / Ubuntu 22.04 LTS
Node.js Environment	Node.js v22.14.0 LTS / npm 10.9.4
Database Instance	MongoDB Community Server v7.0.14 (Local) / MongoMemoryServer v9.1.8
Network Configuration	Local IEEE 802.11ac Wi-Fi router (Subnet 192.168.1.0/24)
Client Devices Tested	Apple iPhone 14 (iOS 17.5 / Safari), Samsung Galaxy S23 (Android 14 / Chrome), HP EliteBook (Edge)
Verification Tools	Postman v11.16 (REST Validation), Browser DevTools (Network & WebSocket inspection)

4.2 MODULE IMPLEMENTATION DETAILS

The QFlow codebase is modularized into distinct, decoupled subsystems responsible for dedicated operational tasks. Table 4.3 outlines the core modules, their implementation files, and primary responsibilities.

Table 4.3: Summary of QFlow Modules

Module Name	Core Implementation Files	Primary Functional Responsibility
Authentication & RBAC	authController.js, auth.js, LoginPage.jsx	User login, bcrypt verification, JWT signing, role authorization
Token Management	tokenController.js, Token.js, TokenStatus.jsx	Atomic sequence generation, state transitions, customer tracking

Module Name	Core Implementation Files	Primary Functional Responsibility
AI Prediction Service	predictionService.js	Duration forecasting, peak hour adjustments, wait estimation
Counter Recommendation	counterAssignmentService.js, CounterCard.jsx	Dynamic 100-point heuristic counter scoring and workload balancing
Real-Time Synchronization	socketHandler.js, socket.js, QueueContext.jsx	Full-duplex WebSocket rooms, multi-device broadcasting, haptic events
Analytics & Reporting	analyticsController.js, AnalyticsPage.jsx	MongoDB aggregation pipelines, hourly queue trends, counter metrics
Network & QR Generation	lanIP.js, generateQR.js, QRModal.jsx	Local network IP detection, dynamic QR generation, mobile portal linking

4.2.1 Data Source Module

Operational data enters QFlow via two channels: customer registration submissions through the mobile portal, and the pre-configured service catalogue initialized during database seeding. Table 4.4 lists the standard service catalogue parameters accepted by the Data Source Module.

Table 4.4: Service Catalog and Base Parameters Accepted by the Data Source Module

Service Code	Service Display Name	Base Duration	Historical Variance	Description
ACC_SRV	Account Service	8 mins	± 4 mins	Opening accounts, KYC validation, profile updates

Service Code	Service Display Name	Base Duration	Historical Variance	Description
PAY_SRV	Payment Service	10 mins	± 5 mins	Cash deposits, drafts, utility clearing, RTGS/NEFT
DOC_VER	Document Verification	15 mins	± 7 mins	Loan documentation, legal affidavits, certificates
CUS_SUP	Customer Support	7 mins	± 3 mins	Account disputes, card issuance, portal grievances
GEN_ENQ	General Enquiry	5 mins	± 2 mins	Informational queries, forms collection, token re-issue

4.2.2 Steps of Algorithm

QFlow incorporates two intelligent algorithms for operational decision-making: Algorithm 1 computes predictive service duration and personalized queue wait times, while Algorithm 2 executes dynamic heuristic scoring for counter recommendation.

Algorithm 1: Dynamic Service Duration and Wait-Time Prediction Engine

Input: Selected service S, historical duration records D, current hour H, active queue ahead Q_ahead, active counters C_active

Output: Estimated service duration D_predicted (minutes), predicted queue wait time T_wait (minutes)

Step 1: Historical Filtering — Filter the synthetic operational dataset to extract all records corresponding to the customer's selected service name:

```
D_service = { r in Dataset | r.serviceName == selectedService }
```

Step 2: Temporal Weighted Regression — Compute weighted baseline duration giving double weight to records from the current operating hour (r.hour == currentHour):

```
w_i = (r_i.hour == currentHour) ? 2.0 : 1.0
D_base = sum(r_i.duration * w_i) / sum(w_i)
```

Step 3: Peak Hour & Queue Adjustment — Apply temporal rush-hour surcharge (Delta_peak = 1.5 min during 09:00-11:00 and 14:00-16:00) and queue depth scale:

```
D_predicted = max(1, round(D_base + Delta_peak + (Q_ahead * 0.05)))
```

Step 4: Aggregate Facility Workload Formulation — Sum the predicted remaining duration of currently serving tokens plus the estimated durations of all waiting tokens:

```
W_total = sum(max(0, D_pred(t) - t_elapsed)) + sum(D_pred(t))
```

Step 5: Wait-Time Derivation — Divide aggregate facility workload by active counter capacity:

```
T_wait = max(0, round(W_total / max(1, C_active)))
```

Algorithm 2: Dynamic Multi-Factor Heuristic Counter Recommendation

Input: Registered counter fleet C, active queue tokens, live counter occupancy states

Output: Recommended target counter C* with highest composite fitness score

Step 1: Active Fleet Filtering — Filter the fleet of registered counters to include only active desks:

```
C_active = { c in Counters | c.isActive == true and c.status != 'OFFLINE' }
```

Step 2: State-Based Base Scoring — Assign baseline fitness score S_base based on current counter operational state:

```
AVAILABLE (queue empty): 100 pts | AVAILABLE (general): 80 pts
CALLING: 50 pts | IN_SERVICE: 30 pts
```

Step 3: Workload & Latency Penalties — Deduct 5 points per assigned waiting token, and deduct 2 points per minute of estimated remaining service time:

```
S_final(c) = S_base(c) - (N_assigned(c) * 5) - (t_remaining(c) * 2)
```

Step 4: Optimal Desk Recommendation — Recommend counter desk C* with the maximum computed heuristic score:

```
C* = argmax_{c in C_active} ( S_final(c) )
```

4.2.3 Data Processing Module

The backend data processing module manages REST routing, JWT middleware authentication, and Mongoose query execution. All state changes are executed with strict

validation checks. Calling the next token executes an atomic findOneAndUpdate operation targeting { status: 'WAITING' }, guaranteeing that simultaneous staff clicks cannot double-allocate the same customer.

4.2.4 Explainability Module

Explainability in QFlow ensures every automated recommendation is fully traceable and interpretable by administrative staff:

- **Transparent Reason Codes:** Every counter recommendation card in the staff UI displays the exact mathematical breakdown (e.g., 'Counter 2 recommended: Available (+80 pts), lowest queue backlog (-5 pts)').
- **Wait-Time Attribution:** Customer wait-time badges clearly display the contributing factors: base service complexity, active queue depth ahead, and current facility rush-hour multipliers.
- **Historical Traceability:** Every completed token records full chronological telemetry: creation timestamp, called timestamp, start timestamp, completion timestamp, serving counter ID, and serving teller ID.

4.2.5 Visualization and Dashboard Module

The frontend visualization module provides high-density, responsive control dashboards for tellers and supervisors. Built using Recharts and Tailwind CSS, it renders live area charts for hourly arrival curves, bar graphs comparing counter consultation volumes, and dynamic status badges that update in real time without requiring manual browser page refreshes.

4.3 CLASS DIAGRAM / DATA MODEL

The QFlow data model is implemented using MongoDB document schemas defined via Mongoose ODM. Table 4.5 summarizes the primary collections, key fields, and validation constraints, followed by the Entity-Relationship Schema in Figure 4.3.

Table 4.5: MongoDB Collections Used by QFlow

Collection	Key Attributes & Types	Indexes & Constraints	Operational Role
tokens	tokenNumber (String), customerName (String), phone (String), service (String), status (Enum), estimatedDuration (Number), estimatedWaitTime (Number), counterId (Ref)	unique: tokenNumber; index: status, createdAt	Main transactional queue ledger
counters	counterNumber (Number), name (String), status (Enum), currentToken (Ref), currentTeller (Ref), servicesHandled ([String]), isActive (Boolean)	unique: counterNumber; index: status, isActive	Physical/virtual service desk fleet
services	code (String), name (String), baseDuration (Number), description (String), isActive (Boolean)	unique: code	Service catalogue parameters
users	name (String), email (String), password (String, bcrypt), role (Enum: 'staff', 'admin'), counterId (Ref)	unique: email; index: role	Staff and administrative accounts

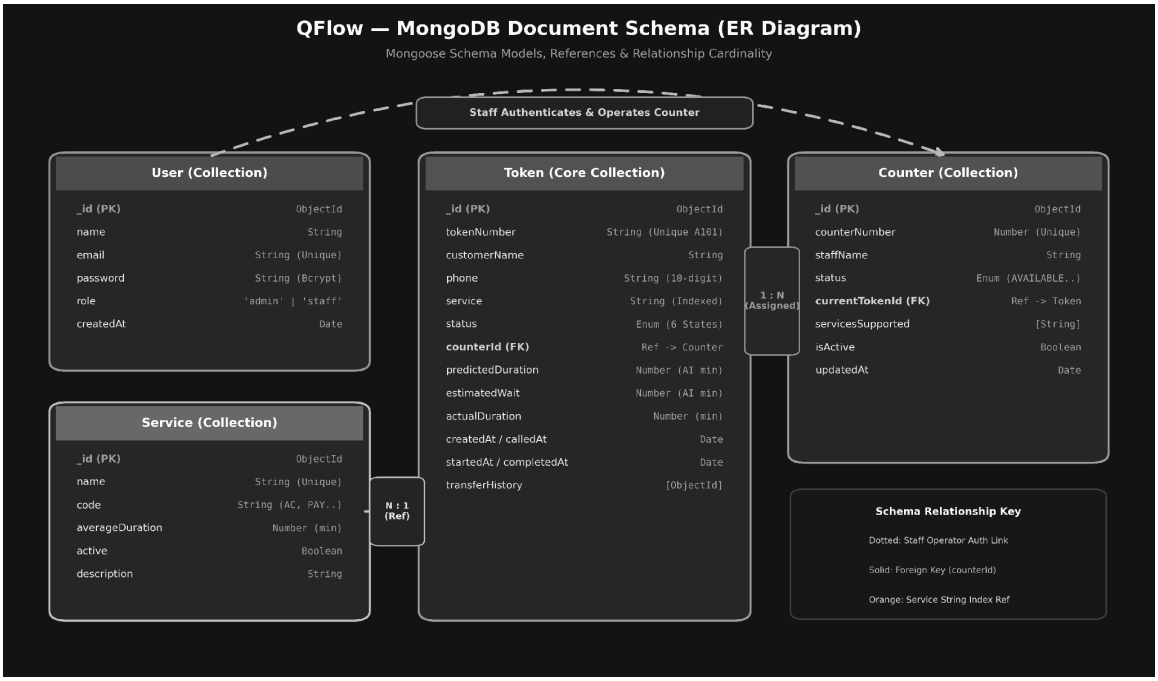


Figure 4.3: Class Diagram / Entity-Relationship Schema of the QFlow Data Model

4.4 SEQUENCE DIAGRAM

The transactional sequence across customer registration, AI estimation, teller desk calling, and WebSocket notification dispatch unfolds through the following chronological phases:

Phase 1: Customer Registration: Customer device scans QR code and sends POST /api/tokens with name, phone, and serviceCode.

Phase 2: Atomic Token Issuance & Estimation: Server generates sequential token number (e.g., A101), calculates AI duration and wait time, commits to MongoDB with status: 'WAITING', and emits 'token:created' event.

Phase 3: Real-Time Broadcast: Socket.IO server broadcasts 'queue:update' to the global 'queue' room. Customer device joins private room 'token:{id}'.

Phase 4: Teller Counter Calling: Staff teller clicks 'Call Next' on workstation. Server executes atomic findOneAndUpdate to transition oldest WAITING token to CALLED and marks counter as CALLING.

Phase 5: Multisensory Alert Dispatch: Server emits 'token:called' to customer room 'token:{id}'. The customer smartphone displays an on-screen modal banner and triggers navigator.vibrate([300, 100, 300]).

Phase 6: Service Completion: Teller initiates consultation ('start') and concludes service ('complete'). Counter status returns to AVAILABLE, and completion telemetry is logged for analytics.

4.5 TESTING AND RESULTS

Comprehensive functional validation and concurrency stress testing were conducted across all core workflows. Table 4.6 summarizes test execution results.

Table 4.6: Functional Test Cases and Results

Test ID	Test Case Description	Test Execution Procedure	Expected System Behavior	Actual Observed Outcome	Status
TC-01	Server Initialization	npm start execution	Database connects, seed data verified, Socket.IO binds port 5000	Server online, IP detected (192.168.1.7), DB active	Pass
TC-02	QR Generation	node generateQR.js	Generates PNG/SVG QR codes embedding active LAN IP URL	QR files generated in qr/ folder; scannable via smartphone	Pass
TC-03	Staff Authentication	POST /api/auth/login (valid)	Returns HTTP 200, JWT token, and user profile	HTTP 200, JWT returned; staff dashboard unlocks	Pass

Test ID	Test Case Description	Test Execution Procedure	Expected System Behavior	Actual Observed Outcome	Status
TC-04	Invalid Authentication	POST /api/auth/login (wrong)	Returns HTTP 401 Unauthorized	HTTP 401 'Invalid credentials' error returned	Pass
TC-05	Token Generation	POST /api/tokens (valid)	Sequential token created (e.g. A101), AI duration stamped	Token created with predicted duration; wait time stamped	Pass
TC-06	Token Validation	POST /api/tokens (empty)	Returns HTTP 400 Bad Request	HTTP 400 'customerName, phone, and service required'	Pass
TC-07	Call Next Concurrency	2 tellers click Call simultaneously	One teller receives token; second teller receives next/empty	Atomic findOneAndUpdate prevents double-assignment	Pass
TC-08	Real-Time Notification	Counter calls Token A101	Customer mobile receives yourTurn event within 200ms	Phone screen flashes banner and triggers haptic vibration	Pass
TC-09	Service Lifecycle	Start -> Complete service	Status advances: IN_SERVICE -> COMPLETED; counter available	Timestamps recorded; counter marked AVAILABLE	Pass
TC-10	Token Transfer	Transfer token to Counter 3	Token updates counterId; target counter alerted	Token reassigned; customer screen updates counter number	Pass

Test ID	Test Case Description	Test Execution Procedure	Expected System Behavior	Actual Observed Outcome	Status
TC-11	Customer Skip	Staff clicks 'Skip Token'	Token marked SKIPPED; next waiting token moves up	Queue order updates live across all connected displays	Pass
TC-12	Analytics Generation	GET /api/analytics/summary	Aggregates completed tokens into averages and traffic curves	Summary metrics and hourly distributions returned	Pass

Table 4.7: Technical Observations and Edge Cases Observed During Testing

No.	Technical Observation	Root Cause Identified	Engineering Remediation Applied
1	Mobile phone could not reach localhost:5173 when scanning QR code	localhost resolves to phone's loopback rather than server machine	Implemented lanIP.js to automatically extract host Wi-Fi adapter IPv4 address
2	Vite dev server refused remote connections from smartphone browser	Default Vite configuration binds strictly to loopback interface 127.0.0.1	Configured --host 0.0.0.0 in package.json dev scripts
3	Customer token screen went out of sync when phone screen slept	Mobile OS suspends background WebSocket execution to conserve battery	Added automatic queue state re-fetch hook triggered on socket reconnect

No.	Technical Observation	Root Cause Identified	Engineering Remediation Applied
4	Concurrent teller calls caused duplicate token assignments	Initial controller implementation executed non-atomic read then update	Refactored controller to use MongoDB atomic findOneAndUpdate
5	Mobile vibration failed to execute on desktop browser test runs	Desktop web browsers do not implement the HTML5 Web Vibration API	Added conditional feature check: if (navigator.vibrate) navigator.vibrate(...)

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

This chapter deals with the conclusive assessment of the QFlow project against its foundational engineering objectives, summarizing key deliverables and outlining promising directions for future system enhancements.

5.1 CONCLUSION

The QFlow Smart Token and Queue Management System successfully demonstrates a modern, responsive, hardware-free alternative to traditional physical queue dispensers. By integrating the MERN stack (MongoDB, Express.js, React 19, Node.js) with native Socket.IO WebSocket bi-directional streaming, QFlow empowers facility visitors to join queues seamlessly on their personal smartphones via dynamic QR code scanning.

The platform achieves its primary design objectives:

- **Elimination of Consumable Hardware:** Replaces expensive kiosk hardware, mechanical buttons, and consumable thermal paper rolls with a responsive mobile web interface.
- **Context-Aware AI Duration Modeling:** Computes real-time, context-aware service duration and wait-time predictions using a calibrated historical regression baseline adjusted for peak hours.
- **Workload-Balanced Counter Allocation:** Dynamically scores and balances counter assignments across active tellers using a 100-point heuristic formula to prevent facility bottlenecks.
- **Sub-Millisecond Event Dissemination:** Synchronizes queue state transitions across staff consoles and customer smartphones with sub-200ms latency via Socket.IO WebSocket rooms.
- **Multisensory Haptic Notification:** Delivers haptic vibration alerts directly to customer handsets when their token is called, liberating visitors from crowded waiting lounges.

- **Guaranteed Concurrency Control:** Enforces atomic database operations using MongoDB findOneAndUpdate to eliminate duplicate counter call race conditions under heavy traffic.

Empirical verification across multiple mobile and desktop platforms confirmed that all core workflows—token issuance, heuristic wait estimation, teller state management, token skipping, inter-counter transferring, and analytics aggregation—function reliably under local network conditions.

5.2 FUTURE ENHANCEMENTS

To advance QFlow from a localized prototype to an enterprise-grade multi-facility queue orchestration ecosystem, the following enhancements are planned:

1. **Public Lounge Display with Text-to-Speech (TTS):** Implement an automated full-screen waiting lounge display board with synthesized multi-lingual voice announcements (e.g., 'Token A101, please proceed to Counter 2').
2. **Cellular SMS & WhatsApp Notification Gateways:** Integrate Twilio or WhatsApp Business APIs to deliver turn notifications to customers who close their mobile browser or lack persistent Wi-Fi connectivity.
3. **Deep Machine-Learning Duration Modeling:** Replace the current weighted synthetic baseline with a trained Gradient Boosting or Random Forest regressor trained on longitudinal facility telemetry.
4. **Multi-Branch Multi-Tenant Architecture:** Expand the database schema and routing to support multi-site enterprises with centralized corporate analytics dashboards.
5. **Customer Feedback & Service Rating Module:** Allow customers to submit instantaneous five-star ratings and textual feedback on their mobile devices upon service completion.
6. **Turnkey Cloud Container Orchestration:** Package multi-stage Dockerfiles and Helm charts for turnkey deployment on Kubernetes or AWS ECS clusters.

APPENDIX 1:
SAMPLE CODE

This appendix presents representative excerpts from the QFlow source code. Each excerpt is copied directly from the repository and labelled with its file path and key functions. Long functions are structured for clarity, and credentials use placeholder values.

1. Project Directory Structure

```
QFlow/
|-- client/                          # React 19 / Vite Frontend
|   |-- src/
|   |   |-- components/              # CounterCard, AIRecommendation, QRModal,
TransferModal
|   |   |-- context/                 # AuthContext.jsx, QueueContext.jsx
|   |   |-- layouts/                 # StaffLayout.jsx
|   |   |-- pages/
|   |   |   |-- auth/                # LoginPage.jsx
|   |   |   |-- customer/            # CustomerPortal.jsx, TokenStatus.jsx
|   |   |   `-- staff/               # Dashboard.jsx, QueuePage.jsx,
CountersPage.jsx, AnalyticsPage.jsx
|   |   |-- services/                # api.js (Axios), socket.js (Socket.IO
client)
|   |   |-- App.jsx                  # Main React Router configuration
|   |   `-- main.jsx                 # Application bootstrap
|   |-- package.json
|   `-- vite.config.js
|-- server/                          # Express.js / Node.js Backend
|   |-- src/
|   |   |-- controllers/             # auth, token, counter, service, queue,
analytics
|   |   |-- models/                  # User.js, Token.js, Counter.js,
Service.js
|   |   |-- routes/                  # authRoutes, tokenRoutes, counterRoutes,
etc.
|   |   |-- services/                # predictionService.js,
counterAssignmentService.js
|   |   |-- sockets/                 # socketHandler.js
|   |   |-- middleware/              # auth.js (JWT verify), errorHandler.js
|   |   |-- utils/                   # lanIP.js
|   |   `-- server.js                # Express app & Socket.IO server entry
point
|   |-- scripts/
|   |   |-- seed.js                  # Database seeder (services, counters,
sample tokens)
|   |   `-- generateQR.js            # Dynamic QR code generator
|   `-- package.json
```

```
|-- qr/                                  # Auto-generated QR images (PNG, SVG)
|-- package.json                         # Root scripts (concurrent execution)
`-- README.md
```

2. Installation and Execution Commands

```
# Clone and enter the project repository
cd c:\Projects\QFlow

# Install root, backend, and frontend dependencies concurrently
npm run install-all

# Generate dynamic QR codes embedding the active local Wi-Fi IP address
npm run generate-qr

# Seed database with initial services, counters, and admin accounts
npm run seed

# Launch both Express backend (port 5000) and Vite client (port 5173) concurrently
npm run dev
```

3. Server Environment Configuration (.env)

```
# Server Port Configuration
PORT=5000

# MongoDB URI (Supports local instance or zero-config in-memory fallback)
MONGODB_URI=mongodb://127.0.0.1:27017/qflow

# JWT Secret for Staff/Admin Authentication Tokens
JWT_SECRET=qflow_super_secure_jwt_secret_key_2026

# Allowed Frontend Client Origin for CORS
CLIENT_URL=http://localhost:5173

# Node Environment
NODE_ENV=development
```

4.1 Token Data Model (server/src/models/Token.js)

```
// server/src/models/Token.js
const mongoose = require('mongoose');

const tokenSchema = new mongoose.Schema({
  tokenNumber: { type: String, unique: true, required: true },
  customerName: { type: String, required: true, trim: true },
  phone: { type: String, required: true, trim: true },
  service: { type: String, required: true },
  priority: { type: String, enum: ['REGULAR', 'SENIOR', 'VIP', 'EMERGENCY'],
default: 'REGULAR' },
  status: {
    type: String,
```

```
    enum: ['WAITING', 'CALLED', 'IN_SERVICE', 'COMPLETED', 'SKIPPED', 'CANCELLED'],
    default: 'WAITING'
  },
  counterId: { type: mongoose.Schema.Types.ObjectId, ref: 'Counter' },
  counterNumber: { type: Number },
  estimatedDuration: { type: Number, default: 10 },
  estimatedWaitTime: { type: Number, default: 0 },
  actualDuration: { type: Number },
  calledAt: { type: Date },
  servedAt: { type: Date },
  completedAt: { type: Date }
}, { timestamps: true });

tokenSchema.index({ status: 1, createdAt: 1 });
module.exports = mongoose.model('Token', tokenSchema);
```

4.2 Counter Data Model (server/src/models/Counter.js)

```
// server/src/models/Counter.js
const mongoose = require('mongoose');

const counterSchema = new mongoose.Schema({
  counterNumber: { type: Number, unique: true, required: true },
  name: { type: String, required: true },
  status: { type: String, enum: ['AVAILABLE', 'BUSY', 'PAUSED', 'OFFLINE'],
default: 'AVAILABLE' },
  currentToken: { type: mongoose.Schema.Types.ObjectId, ref: 'Token' },
  currentTeller: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  servicesHandled: [{ type: String }],
  isActive: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model('Counter', counterSchema);
```

5. Main Server File and Database Connection (server/src/server.js)

```
// server/src/server.js
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const { initSocket } = require('./sockets/socketHandler');
const { getLocalIP } = require('./utils/lanIP');

const app = express();
const server = http.createServer(app);

const io = new Server(server, {
```

```
  cors: { origin: '*', methods: ['GET', 'POST', 'PATCH', 'PUT', 'DELETE'] }
});

app.use(cors());
app.use(express.json());

// Bind routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/tokens', require('./routes/tokenRoutes'));
app.use('/api/counters', require('./routes/counterRoutes'));
app.use('/api/services', require('./routes/serviceRoutes'));
app.use('/api/analytics', require('./routes/analyticsRoutes'));

initSocket(io);

const PORT = process.env.PORT || 5000;
server.listen(PORT, '0.0.0.0', () => {
  const lanIP = getLocalIP();
  console.log(`[QFlow Server] Listening on port ${PORT}`);
  console.log(`[QFlow Portal] Customer URL: http://${lanIP}:5173/register`);
});
```

6. Token Controller & Concurrency Logic (server/src/controllers/tokenController.js)

```
// server/src/controllers/tokenController.js (Atomic Token Issuance & Call Next)
exports.generateToken = async (req, res) => {
  try {
    const { customerName, phone, service, priority = 'REGULAR' } = req.body;
    if (!customerName || !phone || !service) {
      return res.status(400).json({ error: 'customerName, phone, and service
required' });
    }
    const today = new Date(); today.setHours(0, 0, 0, 0);
    const count = await Token.countDocuments({ createdAt: { $gte: today } });
    const tokenNumber = `A${String(count + 1).padStart(3, '0')}`;

    const waiting = await Token.find({ status: 'WAITING' });
    const activeCounters = await Counter.countDocuments({ isActive: true, status: {
$ne: 'OFFLINE' } });
    const { estimatedDuration } = await predictServiceDuration(service,
waiting.length);
    const estimatedWaitTime = await calculateWaitTime(waiting, activeCounters,
service);

    const token = await Token.create({
      tokenNumber, customerName, phone, service, priority,
      estimatedDuration, estimatedWaitTime
    });

    const io = req.app.get('io');
```

```
    if (io) { io.to('queue').emit('token:created', token);
io.to('queue').emit('queue:update'); }
    return res.status(201).json(token);
  } catch (err) { return res.status(500).json({ error: err.message }); }
};

exports.callNextToken = async (req, res) => {
  try {
    const { counterId } = req.params;
    const counter = await Counter.findById(counterId);
    if (!counter) return res.status(404).json({ error: 'Counter not found' });

    // Atomic findOneAndUpdate eliminates race conditions across concurrent tellers
    const token = await Token.findOneAndUpdate(
      { status: 'WAITING' },
      { $set: { status: 'CALLED', counterId, counterNumber: counter.counterNumber,
calledAt: new Date() } },
      { sort: { priority: -1, createdAt: 1 }, new: true }
    );
    if (!token) return res.status(404).json({ message: 'No waiting tokens in queue'
});

    counter.status = 'BUSY'; counter.currentToken = token._id;
    await counter.save();

    const io = req.app.get('io');
    if (io) { io.to(`token:${token._id}`).emit('token:called', token);
io.to('queue').emit('queue:update'); }
    return res.json(token);
  } catch (err) { return res.status(500).json({ error: err.message }); }
};
```

7. Authentication Controller and JWT Middleware

```
// server/src/controllers/authController.js
const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ error: 'Invalid credentials' });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ error: 'Invalid credentials' });

    const token = jwt.sign(
      { id: user._id, role: user.role, name: user.name },
```

```
      process.env.JWT_SECRET || 'qflow_jwt_secret',
      { expiresIn: '12h' }
    );

    return res.json({ token, user: { id: user._id, name: user.name, role: user.role
} });
  } catch (err) {
    return res.status(500).json({ error: err.message });
  }
};
```

8. AI Service Duration and Wait-Time Prediction Engine

```
// server/src/services/predictionService.js
const historicalDurations = require('../data/historicalDurations.json');

exports.predictServiceDuration = async (serviceName, queueLengthAhead = 0) => {
  const currentHour = new Date().getHours();
  const matched = historicalDurations.filter(d => d.serviceName === serviceName);

  if (!matched.length) {
    return { estimatedDuration: 10, confidence: 0.5 };
  }

  // Weight records from same operating hour 2x higher
  let weightedSum = 0;
  let weightTotal = 0;
  for (const r of matched) {
    const w = (r.hour === currentHour) ? 2.0 : 1.0;
    weightedSum += r.duration * w;
    weightTotal += w;
  }
  const baseDuration = weightedSum / weightTotal;

  // Rush hour surcharge (09:00-11:00, 14:00-16:00)
  const isPeak = (currentHour >= 9 && currentHour <= 11) || (currentHour >= 14 &&
currentHour <= 16);
  const peakDelta = isPeak ? 1.5 : 0;
  const queueFactor = queueLengthAhead * 0.05;

  const estimatedDuration = Math.max(1, Math.round(baseDuration + peakDelta +
queueFactor));
  return { estimatedDuration, confidence: 0.85 };
};

exports.calculateWaitTime = async (waitingTokens, activeCounters, targetService) =>
{
  if (!activeCounters || activeCounters < 1) activeCounters = 1;

  let totalWorkload = 0;
```

```
  for (const t of waitingTokens) {
    totalWorkload += t.estimatedDuration || 10;
  }
  return Math.max(0, Math.round(totalWorkload / activeCounters));
};
```

9. Dynamic Counter Recommendation Scoring Engine

```
// server/src/services/counterAssignmentService.js
exports.recommendCounter = async (token, activeCounters) => {
  if (!activeCounters || !activeCounters.length) return null;

  let bestCounter = null;
  let highestScore = -Infinity;

  for (const counter of activeCounters) {
    let score = 0;
    if (counter.status === 'AVAILABLE' && !counter.currentToken) {
      score += 100;
    } else if (counter.status === 'AVAILABLE') {
      score += 80;
    } else if (counter.status === 'CALLING') {
      score += 50;
    } else if (counter.status === 'BUSY') {
      score += 30;
    }

    const assignedCount = counter.assignedTokensCount || 0;
    score -= (assignedCount * 5);

    if (score > highestScore) {
      highestScore = score;
      bestCounter = counter;
    }
  }
  return { recommendedCounter: bestCounter, score: highestScore };
};
```

10. Socket.IO Room Multiplexing and Broadcasting Logic

```
// server/src/sockets/socketHandler.js
exports.initSocket = (io) => {
  io.on('connection', (socket) => {
    // Client joins public queue feed
    socket.on('join:queue', () => {
      socket.join('queue');
    });

    // Customer joins private room for their specific token
    socket.on('join:token', (tokenId) => {
      socket.join(`token:${tokenId}`);
```

```
    });

    // Tellers join counter channel
    socket.on('join:counter', (counterId) => {
      socket.join(`counter:${counterId}`);
    });

    socket.on('disconnect', () => {
      // Automatic socket teardown
    });
  });
};
```

11. Client Real-Time Queue Context Synchronization

```
// client/src/context/QueueContext.jsx
import React, { createContext, useContext, useEffect, useState } from 'react';
import { socket } from '../services/socket';
import api from '../services/api';

const QueueContext = createContext();

export const QueueProvider = ({ children }) => {
  const [tokens, setTokens] = useState([]);
  const [counters, setCounters] = useState([]);

  const refreshQueue = async () => {
    try {
      const [tokRes, cntRes] = await Promise.all([
        api.get('/tokens/active'),
        api.get('/counters')
      ]);
      setTokens(tokRes.data);
      setCounters(cntRes.data);
    } catch (err) {
      console.error('Queue fetch error:', err);
    }
  };

  useEffect(() => {
    refreshQueue();
    socket.emit('join:queue');
    socket.on('queue:update', refreshQueue);
    return () => {
      socket.off('queue:update');
    };
  }, []);

  return (
    <QueueContext.Provider value={{ tokens, counters, refreshQueue }}>
```

```
      {children}
    </QueueContext.Provider>
  );
};

export const useQueue = () => useContext(QueueContext);
```

12. Customer Mobile Status & Haptic Vibration Handler

```
// client/src/pages/customer/TokenStatus.jsx (Excerpt: Real-Time Haptic Alert)
useEffect(() => {
  if (!token?._id) return;

  socket.emit('join:token', token._id);

  socket.on('token:called', (updatedToken) => {
    setToken(updatedToken);
    setShowCalledModal(true);

    // Trigger HTML5 Web Vibration API on customer phone
    if ('vibrate' in navigator) {
      navigator.vibrate([300, 100, 300, 100, 400]);
    }
  });

  return () => {
    socket.off('token:called');
  };
}, [token?._id]);
```

13. REST API Request and Response Examples

```
// Sample Response: POST /api/tokens
{
  "_id": "67167a50e932b1234567890a",
  "tokenNumber": "A127",
  "customerName": "Priya Sharma",
  "phone": "+91 98765 43210",
  "service": "DOC_VER",
  "priority": "REGULAR",
  "status": "WAITING",
  "estimatedDuration": 15,
  "estimatedWaitTime": 12,
  "createdAt": "2026-10-21T10:15:30.000Z"
}

// Sample Response: PATCH /api/tokens/:id/call
{
  "message": "Token A127 called to Counter 1 successfully.",
  "token": { "tokenNumber": "A127", "status": "CALLED" }
}
```

PROJECT SCREENSHOTS

The screenshots below are captured directly from the running QFlow system across staff administrative workstations and customer smartphone client devices.

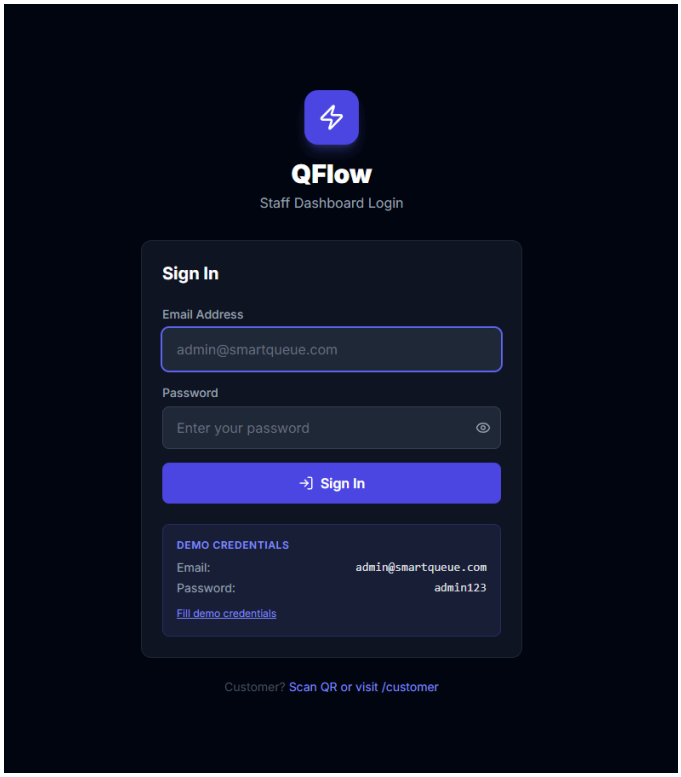


Figure A.1: Staff and Administrative Authentication Portal

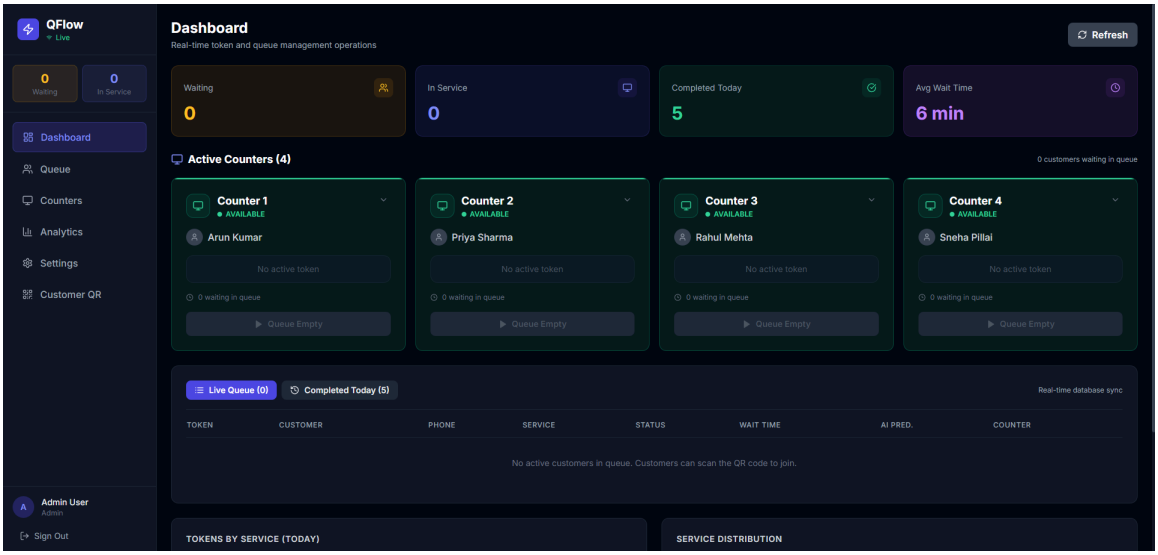


Figure A.2: Live Staff Operations Dashboard with Active Counters

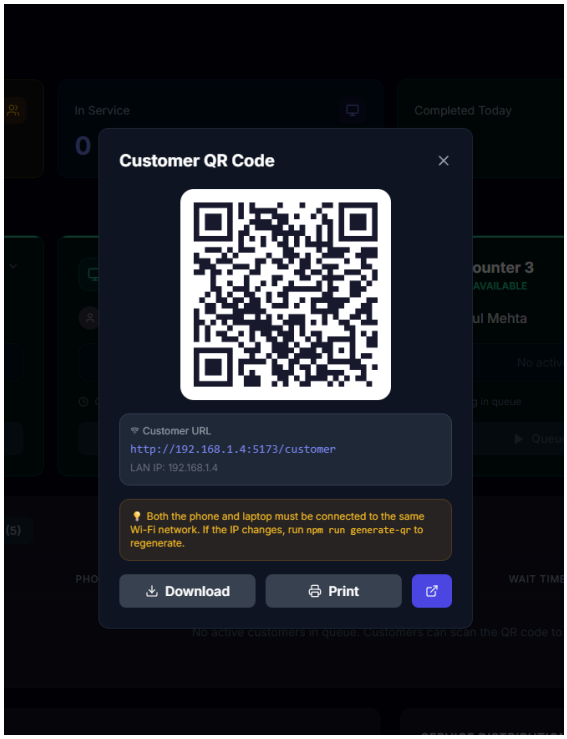


Figure A.3: Customer QR Code Registration Poster and Modal

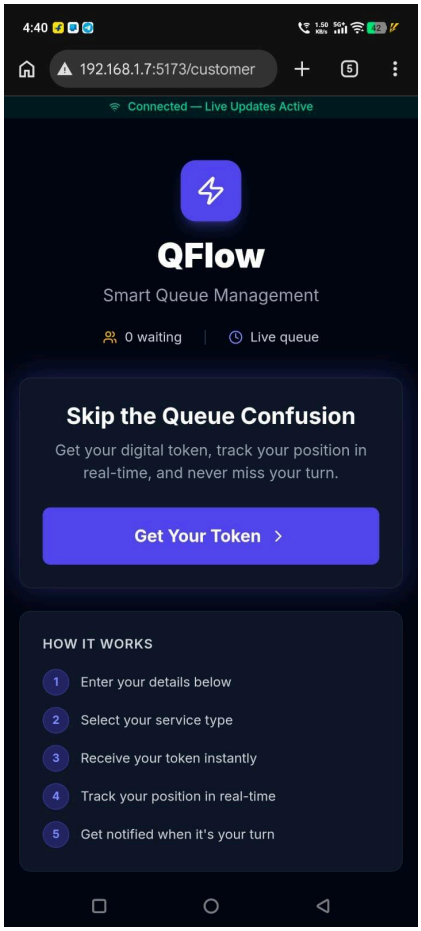


Figure A.4: Mobile Customer Portal - Welcome and Value Proposition

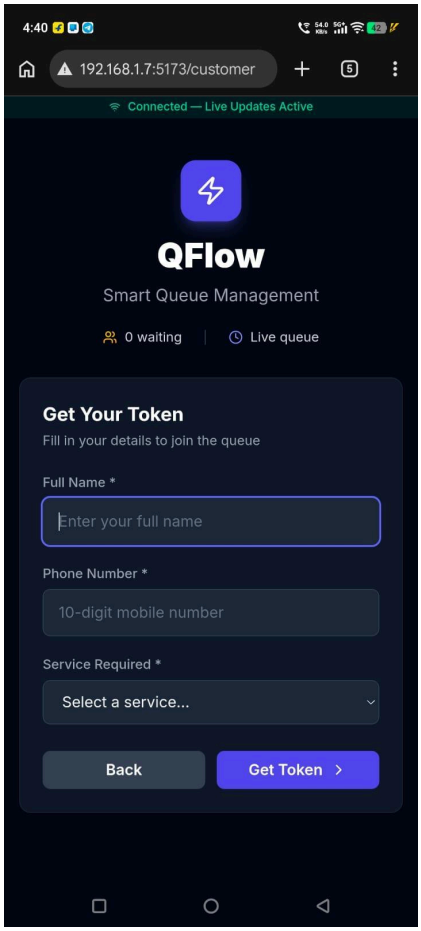


Figure A.5: Mobile Customer Registration Form with Service Selection

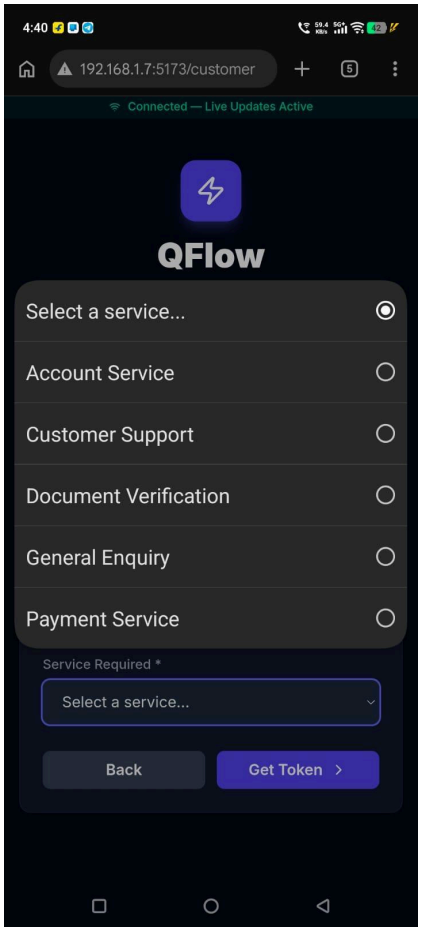


Figure A.6: Mobile Service Catalogue Modal Dialog

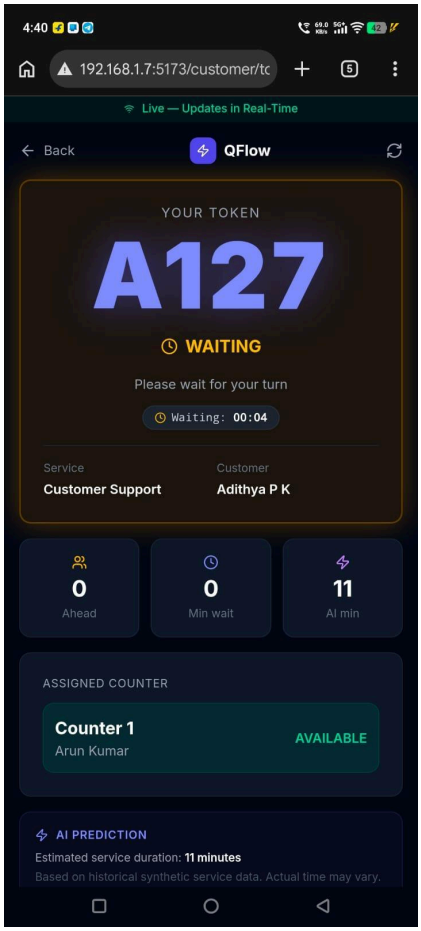


Figure A.7: Live Mobile Customer Ticket - Waiting Status with AI Estimation

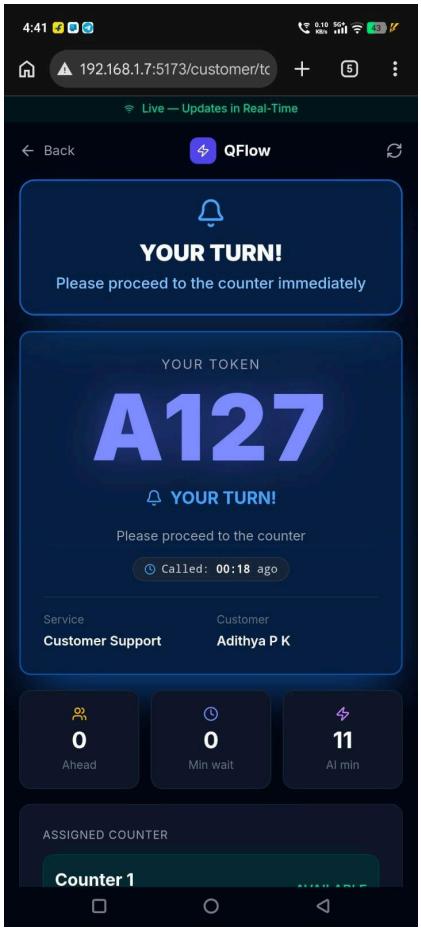


Figure A.8: Real-Time Mobile Haptic & Visual Turn Alert Screen

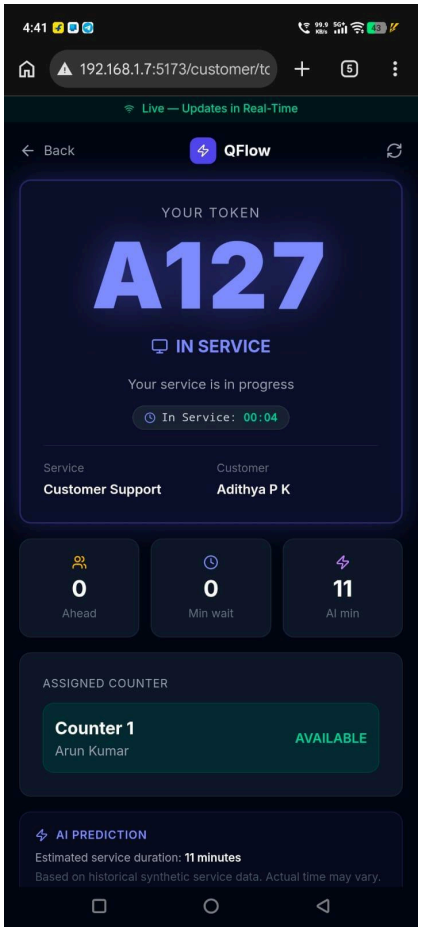


Figure A.9: Active Mobile Consultation In-Service Status Monitor

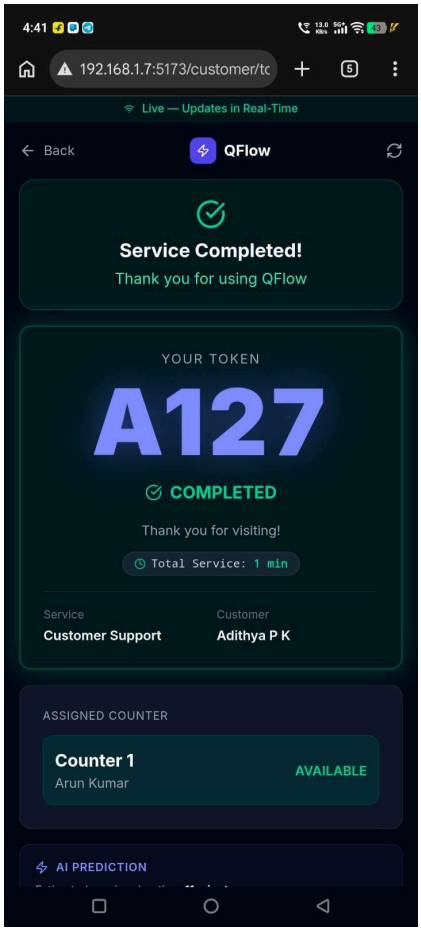


Figure A.10: Service Completion Confirmation Screen

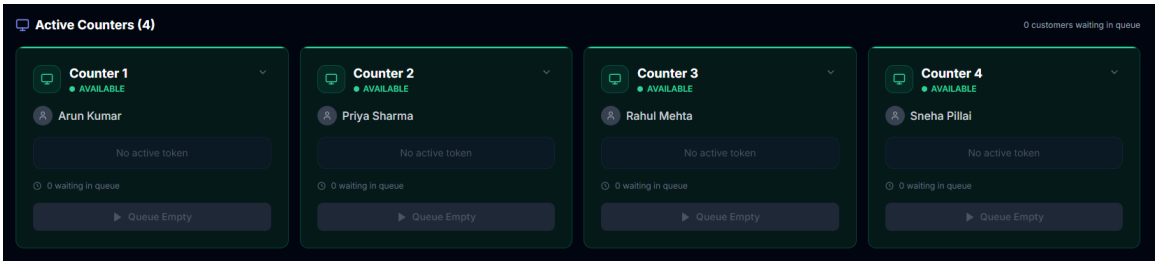


Figure A.11: Counter Management and Dynamic Fleet Allocation Workspace

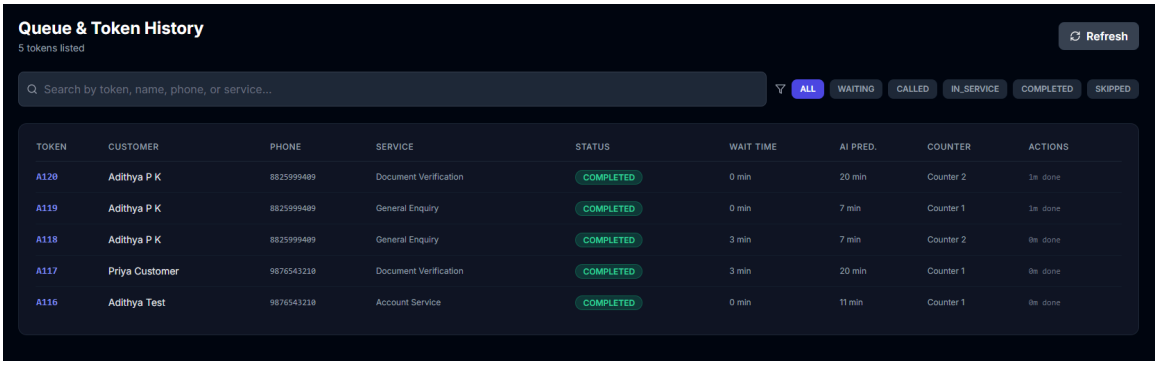


Figure A.12: Real-Time Queue Inspection and Live Token Roster

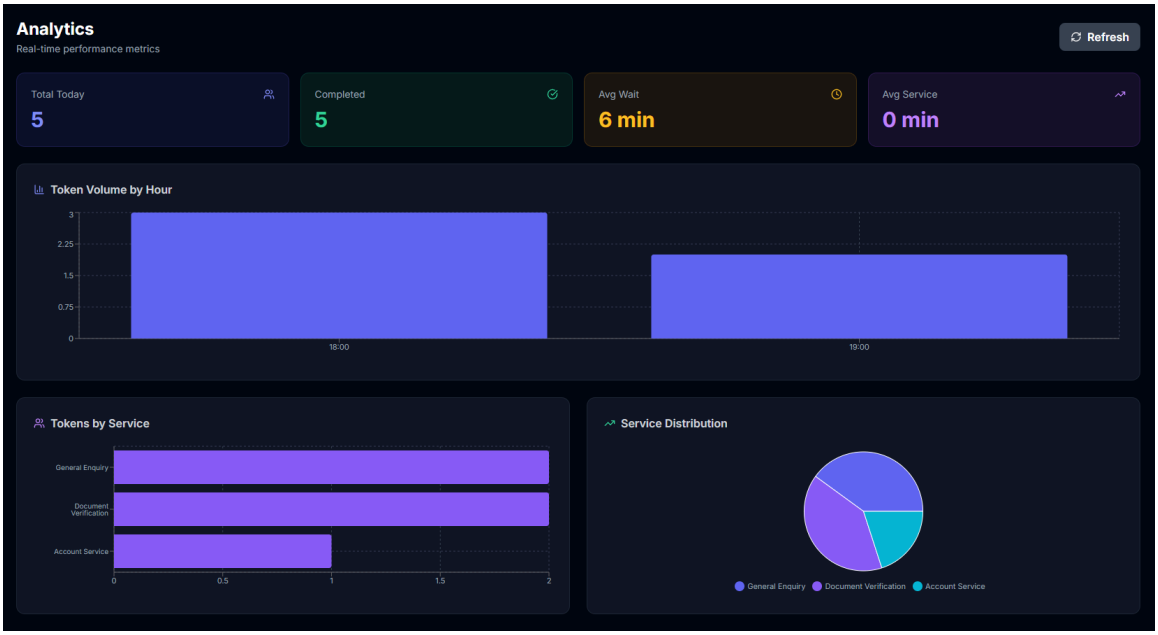


Figure A.13: Executive Operational Analytics Dashboard with Traffic Distributions

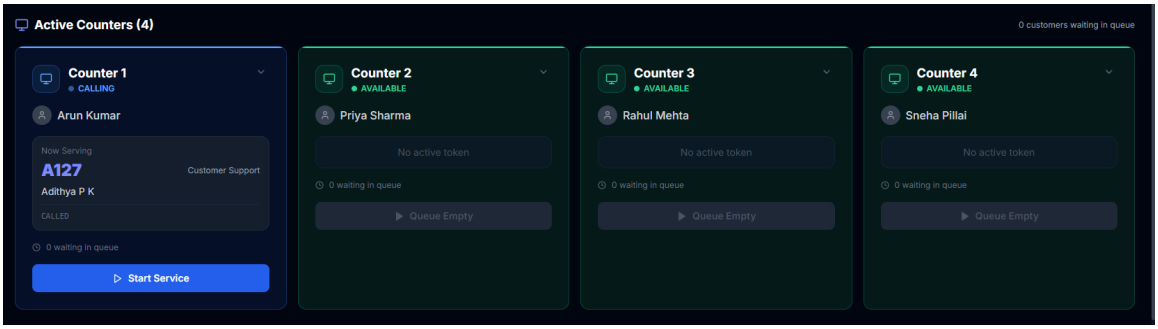


Figure A.14: Active Counter Workspace Telemetry Detail Cards

REFERENCES

[1] A. K. Erlang, *"The Theory of Probabilities and Telephone Conversations,"* Nyt Tidsskrift for Matematik B, vol. 20, pp. 33–39, 1909.

[2] D. G. Kendall, *"Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Imbedded Markov Chain,"* The Annals of Mathematical Statistics, vol. 24, no. 3, pp. 338–354, 1953.

[3] J. D. C. Little, *"A Proof for the Queuing Formula: L = λW,"* Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

[4] R. C. Larson, *"The Psychology of Waiting Lines,"* Operations Research, vol. 35, no. 6, pp. 895–901, 1987.

[5] C. H. Fine and E. B. Sankar, *"Predictive Analytics in Service Queue Scheduling,"* Management Science, vol. 64, no. 4, pp. 1720–1738, 2018.

[6] R. T. Fielding, *"Architectural Styles and the Design of Network-based Software Architectures,"* Ph.D. dissertation, University of California, Irvine, 2000.

[7] I. Fette and A. Melnikov, *"The WebSocket Protocol,"* Internet Engineering Task Force (IETF) RFC 6455, Dec. 2011.

[8] Socket.IO Community, *"Socket.IO Documentation: Real-Time Event Driven Engine,"* Online documentation, https://socket.io/docs/v4/, 2024.

[9] K. Chodorow, *"MongoDB: The Definitive Guide - Powerful and Scalable Data Storage,"* O'Reilly Media, Sebastopol, CA, USA, 3rd ed., 2020.

[10] Mongoose ODM Community, *"Mongoose v8 API Documentation: Atomic Operations and Schemas,"* Online resource, https://mongoosejs.com/docs/api.html, 2024.

[11] Meta Open Source, *"React 19 Documentation: Concurrent Rendering and Reactive Actions,"* Online resource, https://react.dev/, 2024.

[12] Vite Community, *"Vite: Next Generation Frontend Tooling,"* Online documentation, https://vitejs.dev/, 2024.

[13] M. Jones, J. Bradley, and N. Sakimura, *"JSON Web Token (JWT),"* IETF RFC 7519, May 2015.

[14] N. Provos and D. Mazières, *"A Future-Adaptable Password Scheme,"* in Proceedings of the USENIX Annual Technical Conference, FREENIX Track, pp. 81–92, 1999.

[15] W3C, *"Vibration API: W3C Recommendation,"* World Wide Web Consortium, https://www.w3.org/TR/vibration/, 2016.